Greatest common divisor of strings

Q1) What does mean for a string x to "divide" both strings:

it means both str1 and str2 are just x repeated multiple times

str1 = x a (repeated a times)

str2 = x b (repeated b times)

Q2) If such a common divisor x exists at all, what must true about str1 + str2 ?

$$\left.\begin{array}{l} str1 + str2 = xa + xb \\ str2 + str1 = xb + x \end{array}\right\} \text{same}$$

so if str1 + str2 ≠ str2 + str1 return "";

example:

"leet" + "code" ≠ "code" + "leet"

"abcabc" + "abc" = "abc" + "bcabc"

"ababab" + "abab" = "abab" + "ababab"

Q3) Once we know a common divisor exists, what's the largest one?

observation: the largest x has length = GCD(len(str1), len(str2))

Why? think of it mathematically

str1 is x repeated a times → len(str1) must be div by len(x)

str2 is x repeated b times → len(str2) must be div by len(x)

So, len(x) must divide both lengths

↳ the largest such length is GCD

Q4) So what do we actually return?

just the prefix of str1 with length GCD(len(str1), len(str2))

Example

str1 = "ababab", str2 = "abab"
(len = 6) (len = 4)

1) factors of 6 : 1, 2, 3, 6
2) factors of 4 : 1, 2, 4

So, len(x) must appear in both lists

1, 2, 3, 6
1, 2, 4

common factors = 1, 2 ← len(x) can only be there

gcd(6, 4) = 2

Conclusion

we want the **Largest** len(x) that:
- divides len(str1)
- divide len(str2)

That's literally the definition of GCD

GCD(a, b) = Largest number that divides both a and b

GCD using Euclidien algorithm

Keep dividing until remainder is 0. The last non zero remainder is the GCD

GCD(6, 4)

6 / 4 = 1 , remainder = 2

4 / 2 = 2 , remainder = 0 ← Stop!

GCD = 2

GCD(a, b) = GCD(b, a % b)

↳ repeat until remainder = 0

gcd(48, 18)

Step	a	b	remainder (a % b)
1	48	18	12
2	18	12	$\boxed{6}$
3	12	6	0
4	6	0	

gcd(48, 18) = 6

```
func gcd(a, b int) int {
    for b != 0 {
        remainder := a % b
        a = b
        b = remainder
    }
    return a
}
```

```
func gcd(a, b int) int {
    for b != 0 {
        a, b = b, a%b
    }
    return a
}
```